

```

# =====
# BLOB Principe - SAUT 3.6b : EXPOSANTS CRITIQUES (VERSION ETENDUE)
# =====
#
# Ameliorations vs 3.6 :
# 1. Fenetre tau ETENDUE : 0.15 a 1.20 (au lieu de 0.30 a 0.55)
# 2. Resolution variable : dense pres des pics suspectes (0.30-0.50)
# 3. Observables HORS CHAMP : D, S_sim, n_tri, n_tetra, fluctuations
# 4. Run NULL en parallele : graphe Erdos-Renyi sans selection viabilite
# 5. Bootstrap pour erreurs : intervalle de confiance sur chaque grandeur
#
# Methode :
# 3 seeds par tau, N=200, equilibration 60k + mesure 60k (rapide)
# Pour chaque tau, mesure complete : PHI, chi, C, D, S, et leurs variances
# Run NULL : meme protocole mais MCMC sans selection (random pure)
# Comparaison Blob vs Null = signature reelle vs bruit
# =====

import sys
import os

def auto_check():
    print("=" * 70)
    print("  AUTO-TEST CORRUPTION CARACTERES")
    print("=" * 70)
    try:
        if hasattr(sys, 'argv') and len(sys.argv) > 0 and os.path.exists(sys.argv
            source_file = sys.argv[0]
        else:
            source_file = __file__ if '__file__' in globals() else None
        if source_file and os.path.exists(source_file):
            with open(source_file, 'r', encoding='utf-8') as f:
                content = f.read()
            dangerous = {chr(8216): "U+2018", chr(8217): "U+2019",
                chr(8220): "U+201C", chr(8221): "U+201D",
                chr(8211): "U+2013", chr(8212): "U+2014"}
            found = [(name, content.count(c)) for c, name in dangerous.items() if
                if found:
                    print("ERREUR caracteres corrompus :", found)
                    return False
            print("OK : pas de corruption")
        else:
            print("Note : auto-test partiel")
    except Exception as e:

```

```

        print("Note : " + str(e))
    print("=" * 70)
    return True

if not auto_check():
    sys.exit(1)

import numpy as np
import time
import json
import gc
from collections import Counter

KAPPA_0 = 8.0
BETA_VIAB = 0.30
EPS_DECAL = 0.05

OUTPUT_DIR = '/content/blob_saut36b_results'
os.makedirs(OUTPUT_DIR, exist_ok=True)

# =====
# Classe BlobUniverse a N FIXE - mode normal ou null
# =====
class BlobUniverseFixedN:

    def __init__(self, N, tau, seed=0, mode='blob'):
        """mode : 'blob' = selection viabilite, 'null' = swap aleatoire pur"""
        self.rng = np.random.default_rng(seed)
        self.N = N
        self.tau = tau
        self.mode = mode
        self.seed = seed
        self.positions = [self.rng.random(3) for _ in range(N)]
        self.adj = [set() for _ in range(N)]
        self.deg = np.zeros(N, dtype=np.int32)
        self.edge_list = []
        self.n_tri = 0
        self.n_tetra = 0
        self.S_curr = 0.0
        self._initialize_edges()

    def _initialize_edges(self):
        E_target = int(KAPPA_0 * self.N / 2)
        attempts = 0
        max_attempts = E_target * 100

```

```

while len(self.edge_list) < E_target and attempts < max_attempts:
    u = int(self.rng.integers(self.N))
    v = int(self.rng.integers(self.N))
    attempts += 1
    if u == v or v in self.adj[u]:
        continue
    u, v = min(u, v), max(u, v)
    dt_add = len(self.adj[u] & self.adj[v])
    self.adj[u].add(v)
    self.adj[v].add(u)
    self.deg[u] += 1
    self.deg[v] += 1
    self.edge_list.append((u, v))
    self.n_tri += dt_add
    common = self.adj[u] & self.adj[v]
    if len(common) >= 2:
        cl = list(common)
        for i, w in enumerate(cl):
            for x in cl[i+1:]:
                if x in self.adj[w]:
                    self.n_tetra += 1
    self.S_curr = self._compute_S()

def _compute_D(self):
    mu = float(self.deg.mean())
    if mu == 0:
        return 0.0
    return float(self.deg.std()) / mu

def _compute_S(self):
    return np.log(1 + self.n_tri) - BETA_VIAB * self._compute_D()

def _count_tetra_through_edge(self, u, v):
    common = self.adj[u] & self.adj[v]
    if len(common) < 2:
        return 0
    cl = list(common)
    count = 0
    for i, w in enumerate(cl):
        adj_w = self.adj[w]
        for x in cl[i+1:]:
            if x in adj_w:
                count += 1
    return count

def _list_tetrahedra(self):
    result = []

```

```

seen = set()
for u, v in self.edge_list:
    common = self.adj[u] & self.adj[v]
    if len(common) < 2:
        continue
    cl = sorted(common)
    for i, w in enumerate(cl):
        for x in cl[i+1:]:
            if x in self.adj[w]:
                key = tuple(sorted([u, v, w, x]))
                if key not in seen:
                    seen.add(key)
                    result.append(key)
return result

def compute_PHI(self):
    if self.n_tetra == 0:
        return 0.0
    tetras = self._list_tetrahedra()
    if not tetras:
        return 0.0
    edge_to_tetras = {}
    for tet in tetras:
        for i in range(4):
            for j in range(i+1, 4):
                e = (tet[i], tet[j])
                edge_to_tetras.setdefault(e, []).append(tet)
    neighbors_count = {tet: set() for tet in tetras}
    for e, tlist in edge_to_tetras.items():
        if len(tlist) > 1:
            for i, t1 in enumerate(tlist):
                for t2 in tlist[i+1:]:
                    neighbors_count[t1].add(t2)
                    neighbors_count[t2].add(t1)
    total = sum(len(v) for v in neighbors_count.values())
    return float(total) / len(tetras)

def step_swap(self):
    if not self.edge_list:
        return False
    idx = int(self.rng.integers(len(self.edge_list)))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False
    u_add, v_add = min(u_add, v_add), max(u_add, v_add)

```

```

if (u_add, v_add) == (u_rm, v_rm):
    return False
dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
dtetra_rm = self._count_tetra_through_edge(u_rm, v_rm)
self.adj[u_rm].discard(v_rm)
self.adj[v_rm].discard(u_rm)
self.deg[u_rm] -= 1
self.deg[v_rm] -= 1
dt_add = len(self.adj[u_add] & self.adj[v_add])
self.adj[u_add].add(v_add)
self.adj[v_add].add(u_add)
self.deg[u_add] += 1
self.deg[v_add] += 1
dtetra_add = self._count_tetra_through_edge(u_add, v_add)
new_n_tri = self.n_tri - dt_rm + dt_add
if new_n_tri < 0:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False
new_S = np.log(1 + new_n_tri) - BETA_VIAB * self._compute_D()
delta_S = new_S - self.S_curr

# --- DIFFERENCE BLOB / NULL ---
if self.mode == 'null':
    # Acceptation aleatoire pure (pas de selection viabilite)
    accepted = self.rng.random() < 0.5
else: # blob
    log_alpha = self.N * delta_S / self.tau
    accepted = (log_alpha >= 0) or (self.rng.random() < np.exp(max(log_al

if accepted:
    self.edge_list[idx] = (u_add, v_add)
    self.n_tri = new_n_tri
    self.n_tetra += (dtetra_add - dtetra_rm)
    self.S_curr = new_S
    return True
else:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1

```

```

        self.adj[u_rm].add(v_rm)
        self.adj[v_rm].add(u_rm)
        self.deg[u_rm] += 1
        self.deg[v_rm] += 1
        return False

# =====
# Mesure d un point
# =====
def measure_point(N, tau, seed, n_equil, n_measure, mode='blob', n_samples=300):
    sim = BlobUniverseFixedN(N, tau, seed, mode=mode)

    for _ in range(n_equil):
        sim.step_swap()

    sample_interval = max(1, n_measure // n_samples)
    PHI_samples = []
    D_samples = []
    S_samples = []
    n_tri_samples = []
    n_tetra_samples = []

    for step in range(n_measure):
        sim.step_swap()
        if step % sample_interval == 0:
            PHI_samples.append(sim.compute_PHI())
            D_samples.append(sim._compute_D())
            S_samples.append(sim.S_curr)
            n_tri_samples.append(sim.n_tri)
            n_tetra_samples.append(sim.n_tetra)

    PHI_arr = np.array(PHI_samples)
    D_arr = np.array(D_samples)
    S_arr = np.array(S_samples)
    n_tri_arr = np.array(n_tri_samples)
    n_tetra_arr = np.array(n_tetra_samples)

    return {
        'tau': tau, 'seed': seed, 'N': N, 'mode': mode,
        'n_samples': len(PHI_samples),
        'PHI_mean': float(PHI_arr.mean()),
        'PHI_var': float(PHI_arr.var()),
        'PHI_std': float(PHI_arr.std()),
        'D_mean': float(D_arr.mean()),
        'D_var': float(D_arr.var()),
        'S_mean': float(S_arr.mean()),

```

```

        'S_var': float(S_arr.var()),
        'chi': float(N * D_arr.var()),
        'C': float(N * S_arr.var()),
        'n_tri_mean': float(n_tri_arr.mean()),
        'n_tri_var': float(n_tri_arr.var()),
        'n_tetra_mean': float(n_tetra_arr.mean()),
        'n_tetra_var': float(n_tetra_arr.var()),
    }

# =====
# MAIN
# =====

print('')
print('=' * 75)
print('  BLOB - SAUT 3.6b - SCAN TAU ETENDU + NULL')
print('=' * 75)

N_FIXED = int(os.environ.get('BLOB_N', 200))
N_SEEDS = int(os.environ.get('N_SEEDS', 3))
N_EQUIL = int(os.environ.get('N_EQUIL', 60000))
N_MEASURE = int(os.environ.get('N_MEASURE', 60000))
RUN_NULL = os.environ.get('RUN_NULL', '1') == '1'

# Grille tau ETENDUE et raffinee
# Dense entre 0.20 et 0.55 (zone des pics suspectes)
# Plus large jusqu a 1.20 pour voir hors champ
tau_dense = np.linspace(0.20, 0.55, 15) # zone fine
tau_extra = np.array([0.65, 0.80, 1.00, 1.20]) # extension haute
tau_low = np.array([0.15, 0.18]) # extension basse
tau_grid = np.sort(np.concatenate([tau_low, tau_dense, tau_extra]))

print(f'  N fixe : {N_FIXED}')
print(f'  Grille tau ({len(tau_grid)} valeurs) :')
print(f'    extension basse : {tau_low}')
print(f'    zone fine 0.20-0.55 : {tau_dense}')
print(f'    extension haute : {tau_extra}')
print(f'  Seeds : {N_SEEDS}, equilibration {N_EQUIL}, mesure {N_MEASURE}')
print(f'  Run NULL : {RUN_NULL}')
n_modes = 2 if RUN_NULL else 1
total_runs = len(tau_grid) * N_SEEDS * n_modes
duration_per_run_s = (N_EQUIL + N_MEASURE) * 0.001 # ~1 ms par pas
print(f'  Estimation : ~{total_runs * duration_per_run_s / 60:.0f} min total')
print('=' * 75)

t_start = time.time()
all_blob = []

```

```

all_null = []

for tau_idx, tau in enumerate(tau_grid):
    print('')
    print(f' --- tau = {tau:.4f} ({tau_idx+1}/{len(tau_grid)}) ---', flush=True)

    # Run BLOB
    for seed in range(N_SEEDS):
        t0 = time.time()
        m = measure_point(N_FIXED, tau, seed, N_EQUIL, N_MEASURE, mode='blob')
        all_blob.append(m)
        elapsed = time.time() - t0
        print(f' BLOB seed {seed} : PHI={m["PHI_mean"]:.2f} chi={m["chi"]:.4f}')
        gc.collect()

    # Run NULL si active
    if RUN_NULL:
        for seed in range(N_SEEDS):
            t0 = time.time()
            m = measure_point(N_FIXED, tau, seed, N_EQUIL, N_MEASURE, mode='null')
            all_null.append(m)
            elapsed = time.time() - t0
            print(f' NULL seed {seed} : PHI={m["PHI_mean"]:.2f} chi={m["chi"]:.4f}')
            gc.collect()

total_time = time.time() - t_start
print('')
print('=' * 75)
print(f' Toutes mesures terminees en {total_time/60:.1f} min')
print('=' * 75)

# =====
# AGREGATION BLOB
# =====

def aggregate(measurements):
    agg = {}
    for m in measurements:
        tau = m['tau']
        if tau not in agg:
            agg[tau] = {'PHI': [], 'chi': [], 'C': [], 'D': [], 'S': [], 'n_tri':
            agg[tau]['PHI'].append(m['PHI_mean'])
            agg[tau]['chi'].append(m['chi'])
            agg[tau]['C'].append(m['C'])
            agg[tau]['D'].append(m['D_mean'])
            agg[tau]['S'].append(m['S_mean'])
            agg[tau]['n_tri'].append(m['n_tri_mean'])
            agg[tau]['n_tetra'].append(m['n_tetra_mean'])

```

```

        agg[tau]['PHI_var'].append(m['PHI_var'])
    return agg

blob_agg = aggregate(all_blob)
null_agg = aggregate(all_null) if RUN_NULL else {}

tau_sorted = sorted(blob_agg.keys())

# =====
# AFFICHAGE PROFILS BLOB
# =====
print('')
print(' --- PROFILS BLOB (moyenne sur seeds, sigma = std/sqrt(seeds)) ---')
print(f' {"tau":>7s} {"PHI":>10s} {"+/-":>7s} {"chi":>9s} {"+/-":>7s} {"C":>9s}')
print(' ' + '-' * 90)

profiles_blob = {'tau': [], 'PHI': [], 'PHI_err': [], 'chi': [], 'chi_err': [], 'C': [], 'D': [], 'S': [], 'n_tri': [], 'n_tetra': [], 'PHI_var': []}

for t in tau_sorted:
    a = blob_agg[t]
    PHI_m = np.mean(a['PHI'])
    PHI_e = np.std(a['PHI']) / np.sqrt(len(a['PHI']))
    chi_m = np.mean(a['chi'])
    chi_e = np.std(a['chi']) / np.sqrt(len(a['chi']))
    C_m = np.mean(a['C'])
    D_m = np.mean(a['D'])
    n_tri_m = np.mean(a['n_tri'])
    n_tetra_m = np.mean(a['n_tetra'])
    PHI_var_m = np.mean(a['PHI_var'])
    print(f' {t:>7.4f} {PHI_m:>10.2f} {PHI_e:>7.2f} {chi_m:>9.5f} {chi_e:>7.5f}')

    profiles_blob['tau'].append(t)
    profiles_blob['PHI'].append(PHI_m)
    profiles_blob['PHI_err'].append(PHI_e)
    profiles_blob['chi'].append(chi_m)
    profiles_blob['chi_err'].append(chi_e)
    profiles_blob['C'].append(C_m)
    profiles_blob['D'].append(D_m)
    profiles_blob['S'].append(np.mean(a['S']))
    profiles_blob['n_tri'].append(n_tri_m)
    profiles_blob['n_tetra'].append(n_tetra_m)
    profiles_blob['PHI_var'].append(PHI_var_m)

# =====
# AFFICHAGE PROFILS NULL (si actif)
# =====
profiles_null = {}

```

```

if RUN_NULL:
    print('')
    print(' --- PROFILS NULL (random pure, devrait NE PAS dependre de tau) ---')
    print(f' {"tau":>7s} {"PHI":>10s} {"chi":>9s} {"D":>8s} {"n_tri":>8s} {"n_te
    print(' ' + '-' * 70)
    profiles_null = {'tau': [], 'PHI': [], 'chi': [], 'D': [], 'n_tri': [], 'n_te
    for t in tau_sorted:
        if t not in null_agg:
            continue
        a = null_agg[t]
        PHI_m = np.mean(a['PHI'])
        chi_m = np.mean(a['chi'])
        D_m = np.mean(a['D'])
        n_tri_m = np.mean(a['n_tri'])
        n_tetra_m = np.mean(a['n_tetra'])
        print(f' {t:>7.4f} {PHI_m:>10.2f} {chi_m:>9.5f} {D_m:>8.4f} {n_tri_m:>8.
        profiles_null['tau'].append(t)
        profiles_null['PHI'].append(PHI_m)
        profiles_null['chi'].append(chi_m)
        profiles_null['D'].append(D_m)
        profiles_null['n_tri'].append(n_tri_m)
        profiles_null['n_tetra'].append(n_tetra_m)

# =====
# ANALYSE COMPARATIVE BLOB vs NULL
# =====
print('')
print('=' * 75)
print(' ANALYSE COMPARATIVE BLOB vs NULL')
print('=' * 75)

if RUN_NULL and profiles_null:
    print('')
    print(' Pour chaque observable, on compare ratio Blob/Null')
    print(' Ratio ~1 = pas de signal Blob specifique')
    print(' Ratio >>1 ou <<1 = signature Blob')
    print('')
    print(f' {"tau":>7s} {"PHI_b":>9s} {"PHI_n":>9s} {"ratio":>7s} | {"chi_b":>9
    print(' ' + '-' * 95)
    for i, t in enumerate(profiles_blob['tau']):
        if t not in profiles_null['tau']:
            continue
        j = profiles_null['tau'].index(t)
        PHI_b = profiles_blob['PHI'][i]
        PHI_n = profiles_null['PHI'][j]
        chi_b = profiles_blob['chi'][i]
        chi_n = profiles_null['chi'][j]

```

```

D_b = profiles_blob['D'][i]
D_n = profiles_null['D'][j]
ratio_phi = PHI_b / PHI_n if PHI_n > 0 else float('inf')
ratio_chi = chi_b / chi_n if chi_n > 0 else float('inf')
flag = ''
if abs(ratio_phi - 1) > 0.5 or abs(ratio_chi - 1) > 0.5:
    flag = ' <- signal'
print(f'    {t:>7.4f} {PHI_b:>9.2f} {PHI_n:>9.2f} {ratio_phi:>7.3f} | {chi_

# =====
# DETECTION FINE DE STRUCTURE
# =====
print('')
print('=' * 75)
print('  DETECTION FINE DE STRUCTURE')
print('=' * 75)

PHI_arr = np.array(profiles_blob['PHI'])
chi_arr = np.array(profiles_blob['chi'])
tau_arr = np.array(profiles_blob['tau'])

# 1. Pics locaux de chi (différents du bord)
print('')
print('  Pics locaux de chi (signatures de transitions) :')
for i in range(1, len(chi_arr) - 1):
    if chi_arr[i] > chi_arr[i-1] and chi_arr[i] > chi_arr[i+1]:
        # significatif ?
        diff = max(chi_arr[i] - chi_arr[i-1], chi_arr[i] - chi_arr[i+1])
        rel = diff / chi_arr[i] * 100
        print(f'    PIC LOCAL a tau={tau_arr[i]:.4f}, chi={chi_arr[i]:.5f}, promi

# 2. Pente max (point d inflexion)
dPHI = np.diff(PHI_arr) / np.diff(tau_arr)
tau_mid = (tau_arr[:-1] + tau_arr[1:]) / 2
print('')
print(f'  Pente max |dPHI/dtau| a tau={tau_mid[np.argmin(dPHI)]:.4f} (dPHI/dtau={

# 3. Plateau (zones de stabilite)
print('')
print('  Plateaux PHI (|dPHI/dtau| < 1000) :')
for i, (t, d) in enumerate(zip(tau_mid, dPHI)):
    if abs(d) < 1000:
        print(f'    PLATEAU autour de tau={t:.4f} (dPHI/dtau={d:.0f})')

# 4. Fit loi de puissance global
log_PHI = np.log(np.maximum(PHI_arr, 1e-9))
log_tau = np.log(tau_arr)

```

```

slope, intercept = np.polyfit(log_tau, log_PHI, 1)
pred = slope * log_tau + intercept
ss_res = np.sum((log_PHI - pred)**2)
ss_tot = np.sum((log_PHI - log_PHI.mean())**2)
r2 = 1 - ss_res/ss_tot
print('')
print(f' Fit global PHI ~ tau^a : a = {slope:.4f}, R2 = {r2:.4f}')

# =====
# SAUVEGARDE
# =====
report = {
    'config': {
        'N': N_FIXED, 'tau_grid': list(tau_grid),
        'n_seeds': N_SEEDS, 'n_equil': N_EQUIL, 'n_measure': N_MEASURE,
        'run_null': RUN_NULL, 'total_time_s': total_time,
    },
    'profiles_blob': profiles_blob,
    'profiles_null': profiles_null if RUN_NULL else {},
    'all_measurements_blob': all_blob,
    'all_measurements_null': all_null if RUN_NULL else [],
    'global_powerlaw_fit': {'exponent': float(slope), 'r2': float(r2)},
}

report_path = OUTPUT_DIR + '/saut36b_RAPPORT.json'
with open(report_path, 'w') as f:
    json.dump(report, f, indent=2, default=str)
print('')
print(f' Rapport : {report_path}')
print(f' Taille : {os.path.getsize(report_path)} octets')

```